Building ML Pipelines

GeoPython — Quick Reference Handout 04

Linah Ababneh, Ph.D.
LA Research & Consulting

1. What is a Pipeline?

A scikit-learn Pipeline chains preprocessing steps and a model into a single, sequential object. Instead of calling each step manually, you call .fit() and .predict() once — and every step runs in the correct order automatically.

Step	What Happens
1. Define steps	List preprocessing steps + model in order: [("scaler", StandardScaler()), ("model", LinearRegression())]
2. pipe.fit(X_train, y_train)	Scaler learns parameters from training data ONLY; model trains on scaled training data
3. pipe.predict(X_test)	Scaler applies SAME parameters to test data; model predicts — no refitting

2. The Data Leakage Problem

Data leakage occurs when information from the test set is allowed to influence the training process. The most common cause: fitting the scaler on ALL data before splitting.

Approach	Leakage?	Why
Fit scaler on all data → split → train model	✗ YES	Test statistics contaminate scaler parameters
Split first → fit scaler on train only → apply to test	✓ NO	Test data never seen during fitting
Pipeline with .fit(X_train)	✓ NO	Pipeline enforces this automatically

! *A pipeline is the safest way to prevent leakage. If you use a pipeline, you cannot accidentally fit the scaler on the test set — the structure forbids it.*

3. Pipeline Anatomy (Minimal Example)

Code Line	Purpose
from sklearn.pipeline import Pipeline	Import the Pipeline class
from sklearn.preprocessing import StandardScaler	Import a scaler
pipe = Pipeline([("sc", StandardScaler()), ("lr", LinearRegression())])	Build the pipeline with named steps
pipe.fit(X_train, y_train)	Train everything in sequence on training data only
pipe.predict(X_test)	Apply all steps in sequence to test data
pipe.score(X_test, y_test)	Score directly — pipeline handles preprocessing

Named steps (e.g., "sc", "lr") let you access individual components later: pipe["sc"].mean_ gives you the scaler's learned mean — useful for diagnostics.